

Parametric Software Effort Estimation in Generative AI-Assisted Workflows

The Transition from Traditional to AI-Augmented Software Estimation

Traditional software engineering effort estimation models have historically operated under the fundamental assumption that project cost is directly proportional to human keyboard time and manual cognitive labor¹. Classical algorithmic cost models, such as Barry Boehm’s Constructive Cost Model (COCOMO) and early Function Point Analysis (FPA), rely on physical codebase metrics—principally Source Lines of Code (SLOC) or Thousands of Lines of Code (KLOC)—as direct proxies for the overall development lifecycle duration, staffing requirements, and financial budgets². These models were originally calibrated against strictly sequential, waterfall-oriented development phases, where requirements were comprehensively defined prior to codebase construction².

The introduction and rapid adoption of generative artificial intelligence (GenAI) utility tools and autonomous agentic workflows have disrupted these foundational paradigms³. Large Language Models (LLMs) can generate voluminous boilerplate, execute complex scaffolding, translate programming languages, and write extensive unit tests in a fraction of the time required by a human engineer⁷. Consequently, the relationship between physical code volume and human effort has become non-linear and decoupled². While the typing speed and initial file generation phases of the Software Development Life Cycle (SDLC) are dramatically accelerated, developer effort has fundamentally shifted toward contextual verification, code review, architectural alignment, security auditing, and system-level debugging³.

Empirical observations highlight a severe mismatch when applying traditional metrics to AI-assisted workflows³. On highly structured, well-specified, and isolated coding tasks, LLMs can resolve up to 78% of high-complexity features using less than a quarter of the expected human effort³. Conversely, on tasks classified as low-complexity that require non-local context or deep architectural understanding, roughly 22% of implementations demand over 180% of the anticipated human effort due to downstream integration friction, subtle logical hallucinations, and compounding validation cycles³. Therefore, modern estimation engines must analyze the deterministic properties of the proposed code change alongside the cognitive, iterative, and token-based dynamics of human-AI collaboration³.

Metric Category	Traditional Sizing Focus	AI-Native Sizing Focus
Volumetric Metrics	Direct proxy for developer keyboard time and logical	Sizing of context payload; predictive of LLM prompt

	volume ² .	costs and cognitive review overhead ³ .
Change Dispersion	Measures logical layout and the physical breadth of a manual patch ¹⁴ .	Reflects code transformation impact; correlates with context window saturation and model decay ³ .
Logical Branches	Quantifies cyclomatic complexity and testing branch requirements ¹⁶ .	Reflects prompting reasoning complexity and the density of required validation steps ³ .
Test Requirements	Represents standard quality assurance and compliance effort ¹⁹ .	High potential for AI-generated tests; shifts human effort toward debugging failing test assertions ³ .
Deployment Complexity	Sized through system architectures and network routing models ⁴ .	Direct proxy for environment verification and multi-service orchestration bounds ³ .

Deterministic Metrics of the Codebase Delta

To construct a realistic effort estimation model that operates alongside AI assistants, the system must first extract deterministic parameters representing the physical scope of the proposed codebase delta¹¹. These parameters can be derived using static code analysis tools and historical version control metadata¹¹.

File Sizing and Context Dynamics

In manual programming, the physical size of the files being updated relates to the developer's focal attention¹. In AI-assisted workflows, however, file size maps directly to context window saturation and information completeness³. When an LLM modifies a file, it must ingest the entire file—including adjacent classes, imports, and helper methods—to preserve a coherent, syntactically correct global intent³.

If the target file size is large, the interaction incurs substantial token-to-time latency, increases the risk of context decay, and elevates API expenditure³. Large file sizes also introduce a higher "boilerplate tax," which is the repetitive overhead of common imports, license headers, and structural boilerplate²². Open-source counters, such as Ben Boyter's scc (Sloc Cloc and Code)

and Eric S. Raymond's loccount, resolve this by calculating Unique Lines of Code (ULOC) and Logical Lines of Code (LLOC) rather than raw Physical Lines of Code²². This approach normalizes the codebase size by discounting repetitive, generated, or blank lines, yielding a highly accurate metric of the underlying logical volume²².

Delta Sizing and Code Transformation Impact

The scale of a proposed change is determined not only by the size of the target files, but by the magnitude of the delta itself¹⁴. The number of files requiring modification and the exact number of lines to be added or deleted represent the core of the change scope¹⁴.

In an AI-native context, this is defined as the Code Transformation Impact³. Localized modifications confined to a single file or a isolated method typically incur minimal cognitive effort³. However, updates that propagate across multiple files, dependencies, or structural interfaces introduce elevated integration and regression risks³.

Because LLMs have limited capacity to maintain a globally synchronized, intent-aware model of extensive multi-file architectures, high change dispersion often leads to non-local side effects³. These side effects mandate extensive human intervention, turning what seemed like an inexpensive automated generation pass into an expensive manual debugging and correction cycle³.

Codebase Volatility and Historical Hotspots

Software evolution is non-uniform, with certain modules undergoing frequent revisions while others remain functionally stagnant²⁷. Extracting historical change patterns from version control metadata allows the estimation system to identify codebase hotspots²⁷. The open-source git-effort utility (distributed within the git-extras package) parses Git logs to report two key deterministic metrics for every file: the cumulative number of commits and the absolute number of active days those modifications span²¹.

Files exhibiting high change frequency and active days represent areas of high complexity and architectural instability¹⁴. When an AI assistant attempts to modify these hotspots, the probability of introducing regression errors rises³. Consequently, even a trivial line change in a high-volatility file requires a significant safety net of regression testing and developer oversight, which must be factored into the overall effort calculation³.

Mathematical Formalization of Contribution and Architectural Complexity

Rolf-Helge Pfeiffer proposed a formalized algorithm to compute Contribution Complexity (CC), directly assessing the complexity of integration and evolution tasks within Git repositories¹⁴. Pfeiffer's algorithm calculates a score that reflects the difficulty a developer faces when integrating a set of commits into an existing codebase¹⁴.

Step 1: Per-Modification Level Complexity

For each modification m within the set of all modifications M (representing every modified file across the targeted commits), the algorithm extracts four base metrics: the number of lines added ($ml^+(m)$), the number of lines removed ($ml^-(m)$), the number of modified code hunks ($mh(m)$), and the number of modified methods or functions ($mmth(m)$)¹⁴. These physical inputs are mapped to discrete, qualitative complexity scores (ranging from 1 to 5) through specialized step-functions¹⁴:

$$cl^+(l) = cl^-(l) = \begin{cases} 1 \text{ (low)} & \text{if } 0 \leq l \leq 15 \\ 2 \text{ (moderate)} & \text{if } 15 < l \leq 30 \\ 3 \text{ (medium)} & \text{if } 30 < l \leq 60 \\ 4 \text{ (elevated)} & \text{if } 60 < l \leq 90 \\ 5 \text{ (high)} & \text{if } l > 90 \end{cases}$$

$$ch(n) = cmth(n) = \begin{cases} 1 \text{ (low)} & \text{if } 0 \leq n \leq 2 \\ 2 \text{ (moderate)} & \text{if } 2 < n \leq 5 \\ 3 \text{ (medium)} & \text{if } 5 < n \leq 7 \\ 4 \text{ (elevated)} & \text{if } 7 < n \leq 9 \\ 5 \text{ (high)} & \text{if } n > 9 \end{cases}$$

The individual modification complexity ($c_{\text{mod}}(m)$) for a single file is the arithmetic mean of these four sub-scores¹⁴:

$$c_{\text{mod}}(m) = \frac{cl^+(ml^+(m)) + cl^-(ml^-(m)) + ch(mh(m)) + cmth(mmth(m))}{4}$$

Step 2: Aggregating Modification Complexity

To capture the reality that highly complex file modifications scale the integration burden

non-linearly, the algorithm applies an exponential weighting scheme to aggregate individual file complexities¹⁴. It compiles a histogram K of the file-level scores, where j is the frequency of each discrete level $i \in \{1, 2, 3, 4, 5\}$ ¹⁴:

$$K = \text{hist}([c_{\text{mod}}(m) : m \in M])$$

The weighted aggregate modification score ($c_{\text{mod}}^{\forall}$) is calculated using exponential weights (i^i)¹⁴:

$$c_{\text{mod}}^{\forall} = \frac{1}{5} \times \sum_{(i,j) \in K} i^i \times j$$

Step 3: Contribution-Level Metrics Compilation

The algorithm compiles metrics across the entire contribution: the total number of unique files modified (n_{files}), the total number of lines changed (n_{lines}), and the variety of change actions (n_{mk} , which records whether files were added, deleted, renamed, copied, or modified)¹⁴. These represent the physical scope and diversity of the contribution¹⁴.

Step 4: Final Contribution Complexity Formula

The final Contribution Complexity score (c_{contrib}) is computed as the arithmetic mean of four primary sub-complexity scores¹⁴:

$$c_{\text{contrib}} = \frac{1}{4} \times \left(c_{|f|}(n_{\text{files}}) + c_{l/f}\left(\frac{n_{\text{lines}}}{n_{\text{files}}}\right) + c_{\text{mk}}(n_{\text{mk}}) + c_m^{\forall}(c_{\text{mod}}^{\forall}) \right)$$

where $c_{|f|}$ and $c_{l/f}$ use the standard line-modification thresholds, c_{mk} maps the cardinality of modification types directly from 1 to 5, and the overall modification complexity ($c_m^{\forall}$) is mapped via scale boundaries¹⁴:

$$c_m^{\forall}(n) = \begin{cases} 1 \text{ (low)} & \text{if } 0 \leq n \leq 195 \\ 2 \text{ (moderate)} & \text{if } 195 < n \leq 390 \\ 3 \text{ (medium)} & \text{if } 390 < n \leq 781 \\ 4 \text{ (elevated)} & \text{if } 781 < n \leq 1562 \\ 5 \text{ (high)} & \text{if } n > 1562 \end{cases}$$

The final calculated score of c_{contrib} translates back to its nearest corresponding qualitative classification (*low* to *high*)¹⁴. This algorithm provides a deterministic, mathematically rigorous baseline for integration difficulty¹⁴. When combined with AI tools, Pfeiffer's Contribution Complexity helps estimate the human verification effort needed to review automated pull requests³.

Architectural Constraints in Multi-Service Architectures

In multi-service systems, such as microservices or service-oriented applications, estimating effort purely based on individual file modifications is insufficient⁴. Dynamic cross-boundary dependencies often become the dominant source of integration failures and testing overhead³. Open-source static analysis tools like CodeMetrix and pymetrica leverage Abstract Syntax Tree (AST) parsing to evaluate these architectural metrics¹¹. AST-based analysis parses actual code constructs—such as imports, class structures, and function parameters—to compute two primary architectural indicators:

- **Coupling Between Objects (CBO):** CBO measures the degree of interdependence between classes or files by counting unique imports, external type references, parameter types, base class inheritance, and direct object instantiations²⁰.
- **Lack of Cohesion of Methods (LCOM):** LCOM evaluates whether a file contains disparate, unrelated concepts or represents a single logical unit²⁰.

When an update spans multiple services, high CBO across service boundaries indicates that changes will propagate non-locally³. Because AI agents struggle to maintain a global, intent-aware understanding of large, distributed codebases, multi-service tasks with high coupling typically suffer from a higher rate of regression³. The required developer effort must therefore scale exponentially based on the number of interconnected interfaces, even if the total lines of changed code remain minimal³.

Estimating Test Requirements and Edge-Case Densities

A critical component of software effort estimation is quantifying the testing overhead and

identifying potential edge cases¹. In AI-assisted workflows, while writing initial tests is highly accelerated, verifying test completeness and resolving edge cases remain major bottlenecks³.

Structural Predictors of Testing Volume

To determine the number of tests required for a given change, static analysis engines rely on branch detection and control-flow complexity¹⁷. Tools like lizard analyze code complexity without requiring fully compiled environments¹⁶. By parsing control structures such as conditionals, loops, and exception blocks, lizard computes the Cyclomatic Complexity Number (CCN) of the targeted files¹⁶.

The CCN represents the number of linearly independent execution paths through the code¹⁷. Each independent path represents a unique scenario that must be validated¹⁷. Therefore, the cumulative CCN of the proposed changes serves as a direct, deterministic proxy for the minimal number of test cases required to achieve branch coverage¹⁷.

Edge Cases and Non-Fatal Errors

The complexity of a task is heavily influenced by the density of its edge cases¹. While happy-path implementations are easily handled by generative AI tools, edge cases—such as network timeouts, database connection drops, and malformed inputs—often escape the default generation pass of the model³.

Empirical software engineering data reveals that 92% of catastrophic production failures are the result of incorrect handling of non-fatal errors explicitly signaled in the software²⁷.

Furthermore, approximately 77% of these production failures can be reproduced by a standard unit test, highlighting the importance of thorough edge-case validation²⁷.

To estimate the effort required for edge-case validation, the system can parse the target code for exception handlers, conditional boundary checks, and null-safety assertions¹⁷. A higher density of these structures indicates a high-risk area requiring rigorous manual validation and testing, scaling the overall effort estimate³.

Practical Algorithmic Tools and AI-Native Cost Modeling

To operationalize these concepts, engineering teams can leverage existing open-source tools that have redesigned traditional algorithmic cost models to account for the realities of AI-assisted workflows¹⁰.

The Epoch Estimation Engine

Epoch is an open-source time estimation server built on the Model Context Protocol (MCP), designed to connect directly with AI assistants like Claude Code, Cursor, and VS Code¹⁰. Rather than relying on static formulas, Epoch packages established estimation methods—such as PERT, COCOMO II, Monte Carlo simulations, and reference class forecasting—into an interactive toolset¹⁰.



The system includes a bundled reference database containing 126,223 real-world task data points across various task types and complexity levels¹⁰. When the `ai_native` parameter is enabled (which defaults to `true`), Epoch applies correction factors to model the faster velocity of LLM-assisted workflows¹⁰.

For example, a project estimated at 100 person-months under traditional COCOMO II is adjusted to approximately 9 person-months under Epoch's AI-adjusted calculation¹⁰. The system also includes a self-improvement engine that recalibrates its correction factors locally in `~/epoch/` whenever actual task durations are logged, helping correct for systematic estimation biases over time¹⁰.

Ben Boyter's LOCOMO Model

Ben Boyter introduced the LLM Output Cost Model (LOCOMO) in the `scc` static analysis tool, providing a method to estimate the cost and time required to generate a codebase using various LLM configurations¹². LOCOMO focuses on the question: *For this exact codebase, how much would it cost to produce it?*¹²

The model uses the structural complexity of the codebase to estimate the number of prompt-response "cycles" required¹². For example, when validated against Anthropic's open-source C compiler project (which required nearly 2,000 Claude Code sessions and incurred \$20,000 in API costs), LOCOMO maps complexity directly to token consumption and human review overhead¹²:

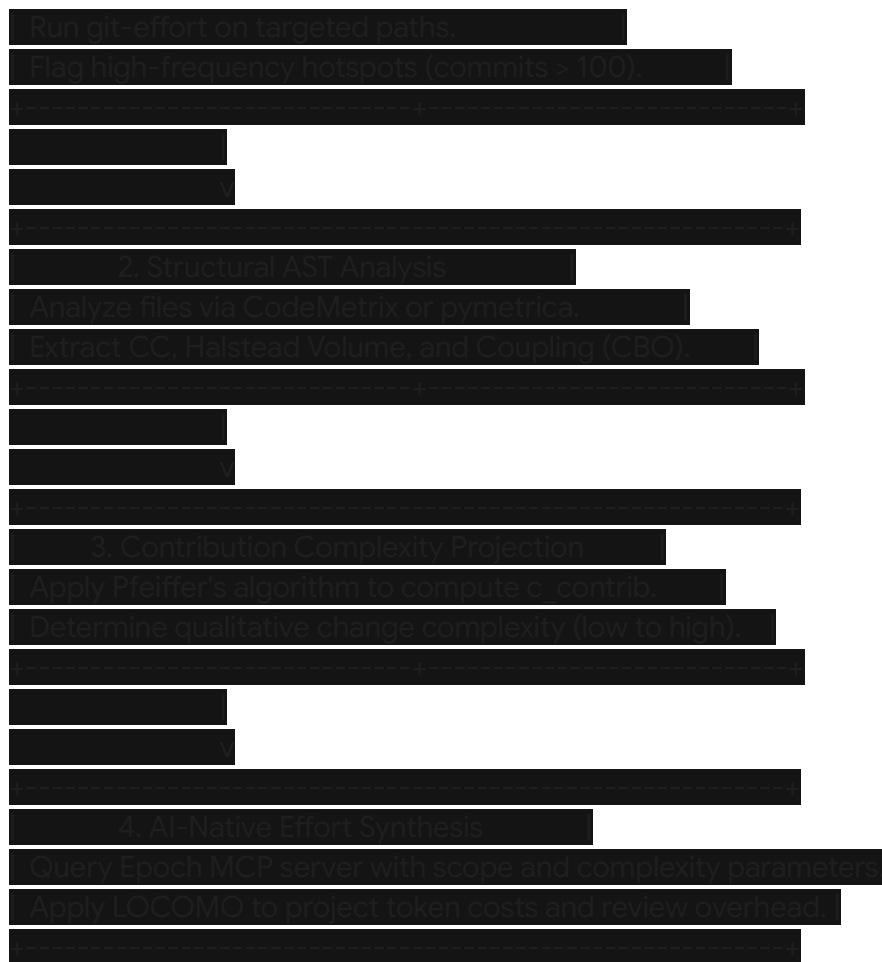
$$\text{Estimated Cycles} = f(\text{Complexity})$$

$$\text{Generation Time} = \frac{\text{Total Tokens}}{\text{Tokens per Second}}$$

$$\text{Human Review Time} = \text{Lines of Code} \times \text{Average Review Rate}$$

This model assumes that a human developer must audit every generated line of code to prevent logical regressions, capturing both the financial cost of API calls and the cognitive overhead of code verification¹².

Tool	Core Sizing Inputs	Algorithmic Base	Key Outputs
scc	Language, LLOC, Complexity, Duplicate files ²⁴ .	COCOMO / LOCOMO (LLM-based) ¹² .	Physical counts, estimated generation cost, and review time ¹² .
Epoch	Task type, complexity, optimistic/pessimistic estimates ¹⁰ .	PERT, COCOMO II (AI-adjusted), Monte Carlo, Reference Class ¹⁰ .	Project duration, schedule risk, API costs, and calibration trends ¹⁰ .
CodeMetrix	Language, AST nodes, Halstead measures ¹¹ .	COCOMO II, FPA, Maintainability index ¹¹ .	Structural complexity, cost estimates, and risk profiling ¹¹ .
lizard	Raw source files, ignore patterns, CCN limits ¹⁶ .	Cyclomatic Complexity, argument counts ¹⁶ .	CCN per function, long method flags, and code clones ¹⁶ .
pymetrica	Python source files, package layers ¹⁷ .	Halstead Volume, CC, Instability, Layer Coupling ¹⁷ .	Instability scores, layer dependencies, and architecture diagrams ¹⁷ .
git-effort	Path, commit	Git commit logs,	Commit volume per



This workflow begins by parsing the historical volatility of the targeted files using git-effort to identify development hotspots²¹. The system then extracts structural metrics—such as Cyclomatic Complexity, Halstead Volume, and Coupling Between Objects—via AST parsing to determine code quality and multi-service dependency risks¹¹.

Next, it applies Pfeiffer's algorithm to calculate the Contribution Complexity score, providing a deterministic measure of integration difficulty¹⁴. Finally, the system inputs these parameters into an AI-native engine like Epoch or LOCOMO¹⁰.

By adjusting the automation ratio (α) to reflect the team's level of AI integration, the engine synthesizes the deterministic physical scope of the change with the estimated cognitive review overhead and API token costs¹⁰. This integrated approach allows engineering leaders to move beyond subjective estimates and establish a data-driven, repeatable framework for planning in the era of AI-assisted software development³.

Referências citadas

1. Software development effort estimation - Wikipedia, https://en.wikipedia.org/wiki/Software_development_effort_estimation

2. COCOMO Model Explained: Formula, Types, and Software Cost Estimation - DataCamp, <https://www.datacamp.com/tutorial/cocomo-model>
3. Toward LLM-aware software effort estimation: a conceptual framework - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC13050940/>
4. Software Supportability Risk Assessment in OT&E (Operational Test and Evaluation): Literature Review, Current Research Review - DTIC, <https://apps.dtic.mil/sti/tr/pdf/ADA191874.pdf>
5. A statistical study of the relevance of lines of code measures in software projects, https://www.researchgate.net/publication/267761547_A_statistical_study_of_the_relevance_of_lines_of_code_measures_in_software_projects
6. (PDF) From COCOMO to GPT: A Comprehensive Evaluation of LLM-Based Software Effort Estimation - ResearchGate, https://www.researchgate.net/publication/401633253_From_COCOMO_to_GPT_A_Comprehensive_Evaluation_of_LLM-Based_Software_Effort_Estimation
7. (PDF) Github Copilot's Impact on Developer Productivity : A Review of Early Evidence, https://www.researchgate.net/publication/397516588_Github_Copilot's_Impact_on_Developer_Productivity_A_Review_of_Early_Evidence
8. Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study - arXiv, <https://arxiv.org/html/2509.20353v2>
9. Developer use cases for AI with GitHub Copilot - Training - Microsoft Learn, <https://learn.microsoft.com/en-us/training/modules/developer-use-cases-for-ai-with-github-copilot/>
10. GitHub - KyaniteLabs/Epoch: Time estimation MCP server for AI agents: PERT, COCOMO II, Monte Carlo, sprint forecasting, token-to-time mapping, cost estimation, and schedule risk tools., <https://github.com/KyaniteLabs/Epoch>
11. CodeMetrix: A sophisticated code analysis and cost estimation tool that provides advanced metrics, quality assessment, and intelligent reporting for software projects. Features COCOMO II modeling, AST-based analysis, and multi-language support. · GitHub, <https://github.com/Ayushx309/codemetrix>
12. Sloc Cloc and Code - LOCOMO (LLM Output COSt MOdel) | Ben E. C. Boyter, <https://boyter.org/posts/sloc-cloc-code-locomo-llm-output-cost-model/>
13. Beyond the Context Window: A Cost-Performance Analysis of Fact-Based Memory vs. Long-Context LLMs for Persistent Agents - arXiv, <https://arxiv.org/html/2603.04814v1>
14. Automatically Assessing Complexity of Contributions to Git Repositories - ITU, https://studwww.itu.dk/~ropf/blog/assets/quatic2021_pfeiffer.pdf
15. (PDF) Developing Adaptive Context Compression Techniques for Large Language Models (LLMs) in Long-Running Interactions - ResearchGate, https://www.researchgate.net/publication/403379915_Developing_Adaptive_Context_Compression_Techniques_for_Large_Language_Models_LLMs_in_Long-Running_Interactions
16. GitHub - terryyin/lizard: A simple code complexity analyser without caring about the C/C++ header files or Java imports, supports most of the popular languages., <https://github.com/terryyin/lizard>

17. GitHub - JuanJFarina/pymetrica: Pymetrica is a Python static analysis tool that computes software engineering metrics such as Cyclomatic Complexity, Halstead Volume, and Maintainability Cost. It analyzes code using the Python AST and provides a CLI for evaluating complexity, maintainability, and architectural stability of Python projects., <https://github.com/JuanJFarina/pymetrica>
18. Technical Debt and Maintainability: How do tools measure it? - ResearchGate, https://www.researchgate.net/publication/358918952_Technical_Debt_and_Maintainability_How_do_tools_measure_it
19. Towards Understanding the Impact of Code Modifications on Software Quality Metrics - arXiv, <https://arxiv.org/html/2404.03953v1>
20. Code Quality Analysis - EmbedOps Docs, https://docs.embedops.io/embedops_cli/how_to/analyze_code_quality/
21. Ask HN: How to understand the large codebase of an open-source project? | Hacker News, <https://news.ycombinator.com/item?id=16299125>
22. Boilerplate Tax - Ranking popular programming languages by density - Ben E. C. Boyter, <https://boyter.org/posts/boilerplate-tax-ranking-popular-languages-by-density/>
23. AaronTraas/loccount: Fork of original at <https://gitlab.com/esr/loccount> so I could make a few quick changes · GitHub - GitHub, <https://github.com/AaronTraas/loccount>
24. GitHub - boyter/scc: Sloc, Cloc and Code: scc is a very fast accurate code counter with complexity calculations and COCOMO estimates written in pure Go, <https://github.com/boyter/scc>
25. Lines of Code (LOC) in Software Engineering - GeeksforGeeks, <https://www.geeksforgeeks.org/software-engineering/lines-of-code-loc-in-software-engineering/>
26. contribution-complexity - PyPI, <https://pypi.org/project/contribution-complexity/>
27. Write tests smarter, not harder - by Maxim Schepelin - Medium, <https://medium.com/booking-com-development/write-tests-smarter-not-harder-fb49c7ab89fd>
28. Automatically Assessing Complexity of Contributions to Git Repositories, <https://pure.itu.dk/en/publications/automatically-assessing-complexity-of-contributions-to-git-repositories/>
29. Developer Productivity with GenAI - arXiv, <https://arxiv.org/html/2510.24265v1>
30. scc — alternative to wc - altbox, <https://altbox.dev/tool/scc/>
31. lizard 1.7.8 - PyPI, <https://pypi.org/project/lizard/1.7.8/>
32. KyaniteLabs - GitHub, <https://github.com/KyaniteLabs>
33. GitHub Copilot and Developer Productivity: An Observational Dose-Response Analysis, <https://arxiv.org/html/2606.00438v1>
34. GitHub Copilot's effect on collaboration has stunned researchers : r/ArtificialIntelligence, https://www.reddit.com/r/ArtificialIntelligence/comments/1ry4l2b/github_copilots_effect_on_collaboration_has/
35. The Fast and Spurious: Developer Productivity with GenAI - arXiv, <https://arxiv.org/html/2510.24265v2>